Light

Algorithms — Searching & Sorting

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Algorithm fundamentals
2. Searching algorithms
3. Sorting algorithms
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **Algorithms: Searching and Sorting (8525 Paper 1)**. This topic covers fundamental algorithms including linear search, binary search, bubble sort, and merge sort. You need to understand how each algorithm works, be able to trace through them, write code implementations, and compare their efficiency.
Exam questions typically ask you to **explain** how algorithms work, **trace** through algorithms using trace tables, **write code** implementations, and **compare** the advantages and disadvantages of different algorithms.

Why algorithms matter

An **algorithm** is a sequence of steps that can be followed to complete a task. Algorithms are fundamental to computing — every program is an implementation of one or more algorithms. Understanding standard algorithms like searching and sorting helps you:

- **Solve problems efficiently:** Different algorithms have different efficiencies — choosing the right one matters
- **Understand program behaviour:** Tracing through algorithms helps you predict and debug code

- **Write better code:** Knowing standard algorithms gives you building blocks for your own programs
- **Compare approaches:** Understanding trade-offs helps you make informed design decisions

Remember: an algorithm is **not** the same as a computer program. An algorithm is a set of instructions that could be followed by a human or a computer. A program is an implementation of an algorithm in a specific programming language.

What you must know

This checklist covers the key exam-visible concepts from the AQA specification. You should be able to explain, trace, and implement each algorithm.

- **Define** the term algorithm and distinguish it from a computer program.
- **Explain** decomposition (breaking problems into sub-problems) and abstraction (removing unnecessary detail).
- **Use trace tables** to determine how algorithms work and what their purpose is.
- **Understand** that more than one algorithm can solve the same problem, and compare their efficiency (time only).
- **Explain** how the linear search algorithm works and write code to implement it.
- **Explain** how the binary search algorithm works and write code to implement it.
- **Compare and contrast** linear and binary search (advantages and disadvantages).
- **Explain** how the bubble sort algorithm works and write code to implement it.
- **Explain** how the merge sort algorithm works.
- **Compare and contrast** bubble sort and merge sort (advantages and disadvantages).

> **Specification note.** AQA Paper 1 assesses algorithms through explanation questions, trace tables, and programming tasks. Exam questions will use AQA pseudo-code, but you can write your answers in Python or pseudo-code. Efficiency comparisons focus on **time** only — you don't need formal Big-O notation.

Toolkit

Algorithm comparison summary

Algorithm	Type	How it works	Best for
Linear search	Searching	Check each item in order until found	Small or unsorted lists
Binary search	Searching	Repeatedly halve the search area	Large sorted lists
Bubble sort	Sorting	Swap adjacent items until in order	Small lists, simple to code
Merge sort	Sorting	Divide, sort halves, merge together	Large lists, consistent speed

Searching algorithms comparison

Feature	Linear search	Binary search
Requires sorted data?	No	Yes
Speed on small lists	Fast	Fast
Speed on large lists	Slow	Very fast
Complexity to code	Simple	More complex
How it works	Checks every item	Eliminates half each time

Sorting algorithms comparison

Feature	Bubble sort	Merge sort
Speed on small lists	Acceptable	Good
Speed on large lists	Very slow	Fast
Memory usage	Low (sorts in place)	Higher (creates new lists)
Complexity to code	Simple	More complex
Consistent speed?	No (varies with data)	Yes (always same speed)

Key definitions

- **Algorithm:** A sequence of steps that can be followed to complete a task.
- **Decomposition:** Breaking a problem into smaller sub-problems, each accomplishing an identifiable task.
- **Abstraction:** The process of removing unnecessary detail from a problem.
- **Efficiency:** How quickly an algorithm completes its task (time efficiency).
- **Trace table:** A table used to track the values of variables as an algorithm executes.

1. Algorithm fundamentals

1.1 What is an algorithm?

An **algorithm** is a sequence of steps that can be followed to complete a task. The key word is "steps" — an algorithm must be a clear, unambiguous set of instructions that can be followed in order.

Important distinction: an algorithm is **not** the same as a computer program. An algorithm is a general solution that could be followed by a human or implemented in any

programming language. A computer program is a specific implementation of an algorithm in a particular language.

For example, a recipe is an algorithm for cooking a dish. The same recipe (algorithm) could be followed by different cooks (implemented in different ways) to produce the same result.

1.2 Decomposition and abstraction

Decomposition means breaking a problem into smaller sub-problems, where each sub-problem accomplishes an identifiable task. This makes complex problems manageable and allows different parts to be solved independently.

For example, to build a game you might decompose it into: display graphics, handle user input, update game state, check for collisions, manage scoring.

Abstraction is the process of removing unnecessary detail from a problem. This helps you focus on what matters and ignore what doesn't. When designing an algorithm, you abstract away implementation details to focus on the logic.

For example, when planning a journey, you might abstract away details like traffic light timings and focus on the route between major points.

1.3 Algorithm efficiency

Different algorithms can solve the same problem, but some are more **efficient** than others. At GCSE, efficiency refers to **time efficiency** — how quickly the algorithm completes its task.

The number of operations an algorithm needs to perform depends on the size of the input. For searching and sorting:

- **Linear search:** In the worst case, checks every item — if the list has 1000 items, it might need 1000 comparisons
- **Binary search:** Halves the search area each time — for 1000 items, needs at most about 10 comparisons
- **Bubble sort:** For a list of n items, may need roughly n^2 comparisons — 1000 items could need nearly 1,000,000 comparisons
- **Merge sort:** Much more efficient for large lists — 1000 items needs roughly 10,000 comparisons

1.4 Trace tables

A **trace table** is used to track the values of variables as an algorithm executes. Each row represents one step of the algorithm, and columns show the values of each variable at that step.

Trace tables help you:

- Understand how an algorithm works step by step
- Predict the output of an algorithm for given inputs
- Find errors (bugs) in algorithms
- Verify that an algorithm produces the correct result

How to complete a trace table:

1. Create columns for each variable and for any outputs
2. Work through the algorithm line by line
3. When a variable changes, write its new value in a new row
4. Only record a value when it actually changes
5. For loops, show each iteration separately
6. Record outputs in the OUTPUT column when they occur

```
EXAMPLE TRACE TABLE

Algorithm:
x ← 5
y ← 3
WHILE x > 0
    y ← y + x
    x ← x - 1
ENDWHILE
OUTPUT y

Trace table:
+------+------+--------+
|  x   |  y   | OUTPUT |
+------+------+--------+
|  5   |  3   |        |
|  5   |  8   |        |
|  4   |  8   |        |
|  4   | 12   |        |
|  3   | 12   |        |
|  3   | 15   |        |
|  2   | 15   |        |
|  2   | 17   |        |
|  1   | 17   |        |
|  1   | 18   |        |
|  0   | 18   |        |
|  0   | 18   |   18   |
+------+------+--------+

Final output: 18
```

1.5 Comparing algorithm efficiency

When comparing algorithms, we consider how the number of operations grows as the input size increases. At GCSE, you only need to understand time efficiency in general terms — no formal notation is required.

List size	Linear search (worst case)	Binary search (worst case)	Bubble sort (worst case)	Merge sort
10 items	10 comparisons	4 comparisons	~100 comparisons	~33 comparisons
100 items	100 comparisons	7 comparisons	~10,000 comparisons	~664 comparisons
1,000 items	1,000 comparisons	10 comparisons	~1,000,000 comparisons	~9,966 comparisons
1,000,000 items	1,000,000 comparisons	20 comparisons	~1,000,000,000,000 comparisons	~19,931,568 comparisons

The table shows why algorithm choice matters for large datasets. Binary search needs only 20 comparisons to search 1 million items, while linear search might need all 1 million. Similarly, merge sort is dramatically faster than bubble sort for large lists.

Worked Example 1 — Defining algorithm terms

Question: Define the term algorithm. [1 mark]

Model answer:
An algorithm is a sequence of steps that can be followed to complete a task **(1)**.

Exam tip: This is a direct specification definition. Learn it word for word — "sequence of steps" and "complete a task" are the key phrases.

Worked Example 2 — Algorithm vs program

Question: Explain the difference between an algorithm and a computer program. [2 marks]

Model answer:
An algorithm is a sequence of steps to complete a task that could be followed by a human or computer **(1)**. A computer program is a specific implementation of an algorithm written in a programming language **(1)**.

Exam tip: The key distinction is that an algorithm is language-independent, while a program is code in a specific language.

Worked Example 3 — Trace table completion

Question: Complete the trace table for the following algorithm when the input is 4. [3 marks]

```
x ← INPUT
total ← 0
FOR i ← 1 TO x
    total ← total + i
ENDFOR
OUTPUT total
```

Model answer:

x	total	i	OUTPUT
4	0		
4	1	1	
4	3	2	
4	6	3	
4	10	4	
4	10		10

Exam tip: Only write a new value when it changes. Work through each line of code systematically. Check your final answer makes sense (1+2+3+4 = 10 ✓).

Worked Example 4 — Decomposition

Question: Define the term decomposition and explain why it is useful when creating algorithms. [2 marks]

Model answer:
Decomposition means breaking a problem into smaller sub-problems **(1)**. This makes complex problems more manageable and allows different parts to be developed and tested separately **(1)**.

> **Exam tip:** Remember: decomposition = breaking down. The benefit is making problems manageable.

2. Searching algorithms

Searching algorithms find a specific item within a collection of data. The two searching algorithms you need to know are **linear search** and **binary search**.

2.1 Linear search

Linear search (also called sequential search) is the simplest searching algorithm. It works by checking each item in a list, one by one, from the beginning until the target item is found or the end of the list is reached.

How linear search works:

1. Start at the first item in the list
2. Compare the current item with the search target
3. If they match, the item has been found — return its position
4. If they don't match, move to the next item
5. Repeat until the item is found or the end of the list is reached
6. If the end is reached without finding the item, it is not in the list

```
LINEAR SEARCH — Finding 7 in [3, 8, 2, 7, 1, 9]

Step 1: Check index 0      [3, 8, 2, 7, 1, 9]
                            ^
                            3 ≠ 7, move on

Step 2: Check index 1      [3, 8, 2, 7, 1, 9]
                               ^
                               8 ≠ 7, move on

Step 3: Check index 2      [3, 8, 2, 7, 1, 9]
                                  ^
                                  2 ≠ 7, move on

Step 4: Check index 3      [3, 8, 2, 7, 1, 9]
                                     ^
                                     7 = 7, FOUND at index 3!

Total comparisons: 4
```

Python implementation:

```python
def linear_search(data, target):
    for i in range(len(data)):
        if data[i] == target:
            return i  # Return position where found
    return -1  # Return -1 if not found

# Example usage
numbers = [3, 8, 2, 7, 1, 9]
result = linear_search(numbers, 7)
print(result)  # Output: 3
```

Advantages of linear search:

- Simple to understand and implement
- Works on unsorted data — no need to sort first
- Works on any data structure that can be iterated
- Efficient for small lists

Disadvantages of linear search:

- Slow for large lists — may need to check every item
- Inefficient compared to binary search on sorted data

2.2 Binary search

Binary search is a more efficient searching algorithm that works by repeatedly dividing the search area in half. It requires the data to be **sorted** first.

Think of binary search like looking up a word in a dictionary. You don't start at page 1 — you open roughly to the middle, see if your word comes before or after that page, then repeat in the relevant half. This is exactly how binary search works.

How binary search works:

1. Find the middle item of the list
2. Compare the middle item with the search target
3. If they match, the item has been found
4. If the target is less than the middle item, search the left half
5. If the target is greater than the middle item, search the right half
6. Repeat until the item is found or the search area is empty

```
BINARY SEARCH — Finding 7 in [1, 2, 3, 5, 7, 8, 9] (sorted)

Index:                        0  1  2  3  4  5  6

Step 1: Find middle          [1, 2, 3, 5, 7, 8, 9]
        low=0, high=6                  ^
        mid=3, value=5                mid
```

```
        5 < 7, so search RIGHT half (indices 4-6)

Step 2: Search right        [1, 2, 3, 5, 7, 8, 9]
        low=4, high=6                       ^
        mid=5, value=8                     mid
        8 > 7, so search LEFT half (index 4 only)

Step 3: Final check         [1, 2, 3, 5, 7, 8, 9]
        low=4, high=4                    ^
        mid=4, value=7                 mid
        7 = 7, FOUND at index 4!

Total comparisons: 3 (vs potentially 5 for linear search)
```

Python implementation:

```python
def binary_search(data, target):
    low = 0
    high = len(data) - 1

    while low <= high:
        mid = (low + high) // 2  # Find middle index

        if data[mid] == target:
            return mid  # Found it!
        elif data[mid] < target:
            low = mid + 1  # Search right half
        else:
            high = mid - 1  # Search left half

    return -1  # Not found

# Example usage
numbers = [1, 2, 3, 5, 7, 8, 9]  # Must be sorted!
result = binary_search(numbers, 7)
print(result)  # Output: 4
```

Advantages of binary search:

- Very efficient for large lists — eliminates half the remaining items each step

- Much faster than linear search on sorted data
- For 1000 items, needs at most about 10 comparisons (vs up to 1000 for linear)

Disadvantages of binary search:

- Data must be sorted first — sorting takes time
- More complex to implement than linear search
- Only works with data structures that allow direct access by index

Worked Example 5 — Explaining linear search

Question: Explain how the linear search algorithm works. [3 marks]

Model answer:
Start at the beginning of the list **(1)**. Compare each item with the search target, moving through the list sequentially **(1)**. If a match is found, return the position; if the end of the list is reached without finding the item, it is not in the list **(1)**.

Exam tip: Key points: start at beginning, check each item sequentially, stop when found or end reached. Don't mention sorted data — linear search doesn't require it.

Worked Example 6 — Explaining binary search

Question: Explain how the binary search algorithm works. [3 marks]

Model answer:
The data must be sorted first **(1)**. Find the middle item and compare it with the target **(1)**. If the target is smaller, search the left half; if larger, search the right half. Repeat until the item is found or the search area is empty **(1)**.

Exam tip: Always mention that data must be sorted. Key concept: repeatedly halving the search area.

Worked Example 7 — Comparing search algorithms

Question: State one advantage and one disadvantage of binary search compared to linear search. [2 marks]

Model answer:
Advantage: Binary search is faster/more efficient for large lists as it eliminates half the data each comparison **(1)**.
Disadvantage: Binary search requires the data to be sorted first, whereas linear search works on unsorted data **(1)**.

Exam tip: The most common comparison points are: speed (binary faster) and sorting requirement (binary needs sorted data).

Worked Example 8 — Writing search code

Question: Write a Python function that uses linear search to find the position of a value in a list. Return -1 if not found. [4 marks]

Model answer:

```python
def find_value(myList, target):
    for i in range(len(myList)):
        if myList[i] == target:
            return i
    return -1
```

Exam tip: Marks typically awarded for: correct loop structure, comparison with target, returning position when found, returning -1 when not found.

3. Sorting algorithms

Sorting algorithms arrange items in a specific order (usually ascending or descending). The two sorting algorithms you need to know are **bubble sort** and **merge sort**.

Sorting is one of the most common tasks in computing. Think about all the times you sort data: alphabetically listing contacts, ordering products by price, ranking search results, or organising files by date. Efficient sorting is essential for many applications, and sorted data enables fast searching with binary search.

3.1 Bubble sort

Bubble sort works by repeatedly comparing adjacent items and swapping them if they are in the wrong order. Larger values "bubble up" to the end of the list with each pass. The name comes from the way larger values gradually rise through the list like bubbles rising in water.

How bubble sort works:

1. Compare the first two items — swap if in wrong order
2. Move to the next pair and compare — swap if needed
3. Continue until the end of the list (this is one "pass")
4. After each pass, the largest unsorted item is in its correct position
5. Repeat passes until no swaps are needed (list is sorted)

```
BUBBLE SORT — Sorting [5, 3, 8, 1, 2]

Pass 1:
[5, 3, 8, 1, 2]  Compare 5,3 → swap  → [3, 5, 8, 1, 2]
[3, 5, 8, 1, 2]  Compare 5,8 → OK    → [3, 5, 8, 1, 2]
[3, 5, 8, 1, 2]  Compare 8,1 → swap  → [3, 5, 1, 8, 2]
[3, 5, 1, 8, 2]  Compare 8,2 → swap  → [3, 5, 1, 2, 8]
End of pass 1: 8 is now in correct position

Pass 2:
[3, 5, 1, 2, 8]  Compare 3,5 → OK    → [3, 5, 1, 2, 8]
[3, 5, 1, 2, 8]  Compare 5,1 → swap  → [3, 1, 5, 2, 8]
[3, 1, 5, 2, 8]  Compare 5,2 → swap  → [3, 1, 2, 5, 8]
End of pass 2: 5 is now in correct position

Pass 3:
[3, 1, 2, 5, 8]  Compare 3,1 → swap  → [1, 3, 2, 5, 8]
[1, 3, 2, 5, 8]  Compare 3,2 → swap  → [1, 2, 3, 5, 8]
End of pass 3: 3 is now in correct position

Pass 4:
[1, 2, 3, 5, 8]  Compare 1,2 → OK    → [1, 2, 3, 5, 8]
No swaps needed — SORTED!

Final result: [1, 2, 3, 5, 8]
```

Python implementation:

```python
def bubble_sort(data):
    n = len(data)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if data[j] > data[j + 1]:
                # Swap adjacent items
                data[j], data[j + 1] = data[j + 1], data[j]
                swapped = True
        if not swapped:
```

```
            break  # Already sorted
    return data

# Example usage
numbers = [5, 3, 8, 1, 2]
print(bubble_sort(numbers))  # Output: [1, 2, 3, 5, 8]
```

Advantages of bubble sort:

- Simple to understand and implement
- Sorts "in place" — doesn't need extra memory for a new list
- Can detect if list is already sorted (no swaps needed)
- Efficient for small lists or nearly sorted data

Disadvantages of bubble sort:

- Very slow for large lists — needs many comparisons
- Inefficient compared to other sorting algorithms
- Performance varies greatly depending on initial order of data

3.2 Merge sort

Merge sort uses a "divide and conquer" approach. It repeatedly divides the list in half until each sub-list has one item, then merges the sub-lists back together in sorted order. Think of merge sort like sorting a deck of cards. First, deal the cards into smaller and smaller piles until each pile has just one card (which is trivially sorted). Then, pick up pairs of piles and merge them together in order, repeatedly, until you have one sorted deck.

How merge sort works:

1. **Divide:** Split the list in half, repeatedly, until each sub-list has one item
2. **Conquer:** A single item is already "sorted"
3. **Merge:** Combine pairs of sub-lists, comparing items to merge them in order
4. Continue merging until the full list is reassembled in sorted order

```
MERGE SORT — Sorting [5, 3, 8, 1, 2, 7]

DIVIDE PHASE (split until single items):

            [5, 3, 8, 1, 2, 7]
                    |
        +----------+----------+
        |                     |
    [5, 3, 8]             [1, 2, 7]
        |                     |
    +---+---+             +---+---+
    |       |             |       |
  [5,3]   [8]           [1,2]   [7]
    |                     |
```

```
  +-+-+                 +-+-+
  |   |                 |   |
 [5] [3]               [1] [2]


MERGE PHASE (combine in sorted order):

 [5] [3]               [1] [2]
  |   |                 |   |
  +-+-+                 +-+-+
    |                     |
  [3,5]   [8]           [1,2]   [7]
    |       |             |       |
    +---+---+             +---+---+
        |                     |
    [3, 5, 8]             [1, 2, 7]
        |                     |
        +----------+----------+
                   |
            [1, 2, 3, 5, 7, 8]

Final result: [1, 2, 3, 5, 7, 8]
```

The merge process explained:
When merging two sorted lists, compare the first item of each list. Take the smaller one and add it to the result. Repeat until both lists are empty.

```
MERGING [3, 5, 8] and [1, 2, 7]:

Lists:    [3, 5, 8]    [1, 2, 7]    Result: []
Compare:   3 > 1, take 1            Result: [1]

Lists:    [3, 5, 8]    [2, 7]       Result: [1]
Compare:   3 > 2, take 2            Result: [1, 2]

Lists:    [3, 5, 8]    [7]          Result: [1, 2]
Compare:   3 < 7, take 3            Result: [1, 2, 3]

Lists:    [5, 8]       [7]          Result: [1, 2, 3]
Compare:   5 < 7, take 5            Result: [1, 2, 3, 5]
```

```
Lists:    [8]          [7]          Result: [1, 2, 3, 5]
Compare:   8 > 7, take 7            Result: [1, 2, 3, 5, 7]

Lists:    [8]          []           Result: [1, 2, 3, 5, 7]
One list empty, take 8              Result: [1, 2, 3, 5, 7, 8]
```

Advantages of merge sort:

- Very efficient for large lists
- Consistent performance — always takes the same time regardless of initial order
- Much faster than bubble sort for large datasets
- Predictable behaviour

Disadvantages of merge sort:

- More complex to understand and implement
- Requires extra memory to store sub-lists during merging
- Slower than bubble sort for very small or nearly sorted lists

Exam technique: Explaining algorithms. When asked to explain how an algorithm works, describe the key steps in order. Use the correct terminology (e.g., "adjacent items", "halve the search area", "merge in sorted order"). For full marks, explain what happens at each stage and mention any requirements (e.g., binary search needs sorted data). Avoid being too vague — "it checks items" is not enough. Be specific about HOW it checks items.

Exam technique: Comparison questions. When comparing algorithms, always give both advantages and disadvantages. Common comparison points include: speed (efficiency), memory usage, complexity to implement, and requirements (e.g., sorted data). Link your answers to the context — if asked about a large dataset, emphasise that efficiency matters more for large data.

Worked Example 9 — Explaining bubble sort

Question: Explain how the bubble sort algorithm works. [3 marks]

Model answer:
Compare adjacent pairs of items and swap them if they are in the wrong order **(1)**. Continue through the list — after each pass, the largest unsorted item moves to its correct position **(1)**. Repeat passes until no swaps are needed, meaning the list is sorted **(1)**.

Exam tip: Key points: compare adjacent pairs, swap if needed, repeat passes until no swaps. The name "bubble" refers to values bubbling to the top.

Worked Example 10 — Explaining merge sort

Question: Explain how the merge sort algorithm works. [4 marks]

Model answer:
Divide the list repeatedly in half until each sub-list contains one item **(1)**. A single item is considered sorted **(1)**. Merge pairs of sub-lists by comparing items and combining them in order **(1)**. Continue merging until the full sorted list is reassembled **(1)**.

Exam tip: Describe the two phases: divide (split to single items) and merge (combine in order). Mention that merging compares items from each sub-list.

Worked Example 11 — Comparing sorting algorithms

Question: Compare the efficiency of bubble sort and merge sort for sorting a large list of 10,000 items. [3 marks]

Model answer:
Bubble sort would be very slow because it may need to make roughly 100,000,000 comparisons (n^2) **(1)**. Merge sort would be much faster, needing roughly 130,000 comparisons **(1)**. However, merge sort requires extra memory to store sub-lists during the merge process, whereas bubble sort sorts in place **(1)**.

Exam tip: For large lists, merge sort is dramatically faster. But mention the trade-off: merge sort uses more memory.

Worked Example 12 — Writing bubble sort code

Question: Write a Python function to sort a list using bubble sort. [5 marks]

Model answer:

```python
def bubble_sort(myList):
    n = len(myList)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if myList[j] > myList[j + 1]:
                temp = myList[j]
```

```
                myList[j] = myList[j + 1]
                myList[j + 1] = temp
    return myList
```

Exam tip: Marks for: nested loops, comparing adjacent items, swapping correctly, reducing range each pass. The swap can use tuple assignment or a temporary variable.

Worked examples summary

The worked examples throughout this topic cover the main exam question styles you'll encounter. Here's a summary of what each example demonstrates:

Example	Topic	Marks	Skill tested
1	Defining algorithm	1	Recall definition
2	Algorithm vs program	2	Compare concepts
3	Trace table completion	3	Trace execution
4	Decomposition	2	Define and explain
5	Explaining linear search	3	Describe algorithm
6	Explaining binary search	3	Describe algorithm
7	Comparing search algorithms	2	Advantages/disadvantages
8	Writing linear search code	4	Programming
9	Explaining bubble sort	3	Describe algorithm
10	Explaining merge sort	4	Describe algorithm
11	Comparing sorting algorithms	3	Efficiency comparison
12	Writing bubble sort code	5	Programming

Quick reference

Key definitions

Term	Definition
Algorithm	A sequence of steps that can be followed to complete a task.

Term	Definition
Decomposition	Breaking a problem into smaller sub-problems, each accomplishing an identifiable task.
Abstraction	The process of removing unnecessary detail from a problem.
Linear search	Checks each item sequentially until the target is found or the end is reached.
Binary search	Repeatedly halves a sorted list to find the target item.
Bubble sort	Compares adjacent items and swaps them if in wrong order; repeats until sorted.
Merge sort	Divides list into single items, then merges them back together in sorted order.

Algorithm comparison at a glance

Algorithm	Requires sorted data?	Speed (large data)	Memory use	Complexity
Linear search	No	Slow	Low	Simple
Binary search	Yes	Fast	Low	Medium
Bubble sort	N/A	Very slow	Low (in place)	Simple
Merge sort	N/A	Fast	Higher (extra lists)	Complex

When to use each algorithm

Use linear search when:

- Data is unsorted
- List is small
- Only searching once
- Simplicity matters most

Use binary search when:

- Data is already sorted
- List is large
- Searching many times
- Speed is important

Use bubble sort when:

- List is small

- Data is nearly sorted
- Memory is limited
- Simple code is needed

Use merge sort when:

- List is large
- Consistent speed needed
- Memory is available
- Performance matters

Typical pitfalls

Pitfall 1 — Saying binary search is always better.
Wrong: "Binary search is better than linear search."
Right: Binary search is faster but requires sorted data. For unsorted data, linear search is the only option.

Pitfall 2 — Forgetting binary search needs sorted data.
Wrong: Using binary search on [5, 2, 8, 1, 9].
Right: Binary search only works on sorted data. Sort first or use linear search.

Pitfall 3 — Confusing algorithm and program.
Wrong: "An algorithm is code written in Python."
Right: An algorithm is a sequence of steps; a program is an implementation in a specific language.

Pitfall 4 — Incomplete trace tables.
Wrong: Only showing final values.
Right: Show every change to every variable, step by step.

Pitfall 5 — Vague algorithm explanations.
Wrong: "It checks through the list."
Right: Describe the specific steps: start at beginning, compare each item, stop when found or end reached.

Pitfall 6 — Forgetting merge sort's divide phase.
Wrong: Only describing the merging process.
Right: Describe both phases: divide into single items, then merge in sorted order.

Pitfall 7 — Wrong bubble sort swap logic.
Wrong: `a = b; b = a` (loses original value of a).
Right: Use temp variable: `temp = a; a = b; b = temp` or tuple swap.

Pitfall 8 — Saying bubble sort is efficient.
Wrong: "Bubble sort is efficient for large lists."
Right: Bubble sort is slow for large lists. It's only efficient for small or nearly sorted data.

Pitfall 9 — Missing the "not found" case.
Wrong: Search function that only returns when item is found.
Right: Always handle the case when the item is not in the list (return -1 or False).

Pitfall 10 — Forgetting merge sort uses extra memory.
Wrong: Saying merge sort has no disadvantages.
Right: Merge sort requires extra memory for sub-lists; bubble sort sorts "in place".

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.